Executable Falsification for AI-Assisted Research: Construction-Matched Controls, a Failure Taxonomy, and a Research Ledger

Giovanni Pezzin
Physis Research Program

Abstract

When AI agents design, implement, execute and interpret their own experiments, the loop that produces results also produces the errors that make those results wrong. We report twelve documented cases from a single research programme in which a positive result was killed by a control, and we extract from them a control taxonomy, a failure taxonomy, and an executable workflow. Our central instrument is the *construction-matched null*: a control generated by a procedure that preserves irrelevant aspects of the experimental construction while selectively destroying or perturbing the mechanism claimed to produce the effect. The distinction matters because our own first definition of it was wrong, and we report that error and its correction. We show a case in which only the null changed — from random injection to nearest-cell injection — and the measured performance moved from AUC 0.882 to 0.506. We then formalise the workflow that produced these catches: claims are registered before running, with a pre-registered null; the null runs in the same execution as the treatment; a verdict is assigned from the result; revisions preserve the superseded claim; and failed branches are retained rather than deleted. We are explicit about what this workflow does *not* do: it records protocol violations but cannot prevent them, and we have no control group showing that it produces better research than research without it.

1 Introduction

An AI agent that designs an experiment, writes the code, runs it, reads the output and writes the conclusion concentrates four normally-separated roles into one loop that executes in minutes. The efficiency is the point. The hazard is that each of those four roles is a place where an error can enter, and a single machine-mediated loop can generate and propagate such errors far faster than a human-paced process.

This paper asks what actually goes wrong, and what instrumentation catches it. It is deliberately not a success story: the most useful thing the programme produced was a series of strong positive results that turned out to be artifacts of their own controls.

We make three contributions, in three layers:

- **A failure catalog** (Section 3): twelve cases, each with the intended claim, the apparent positive, the failure mode, the control that caught it, the corrected result, the control's type, and when in the process it was detected.
- **A control taxonomy** (Section 4): a precise definition of the construction-matched null, and an explicit account of what is *not* one — because stretching the term to cover every good research practice makes it useless.

- **An executable workflow** (Section 5): the claim/experiment/verdict/revision structures, the invariant that no result may modify its own decision rule after observation, and the retention of failed branches, demonstrated on a real lineage.

2 The null is the instrument

Consider the canonical experiment in our programme. A system claims to detect misfiled ontology entries at AUC 0.880. Whether that number means anything depends entirely on where the test errors were placed.

If errors are placed uniformly at random across cells, a large displacement results, and any distance-based scorer detects them. If errors are placed at the entry's nearest alternative cell — a plausible mistake of the kind a person actually makes — the displacement is small and the scorer has little to work with.

We ran both. AUC moved from 0.880 to 0.506. The system was identical. The metric was identical. Only the construction of the null changed, and it changed the conclusion from "working detector" to "chance".

2.1 A definition we got wrong

An earlier draft of this work defined a construction-matched null as one that "places errors where the system cannot detect them." The same draft then stated correctly, in its principles section, that a null placing errors where the system cannot detect them "is not a control; it is a tautology." Both sentences cannot hold.

The second is right. A null whose perturbations are undetectable by construction guarantees the result and tests nothing. The correct definition is:

> A **construction-matched null** is a control generated by a procedure that preserves irrelevant aspects of the experimental construction while selectively destroying or perturbing the mechanism claimed to produce the effect.

The operative words are *preserves* and *selectively*. A valid control must keep everything that is not the claim — the data distribution, the nuisance structure, the difficulty of the task — and vary only the mechanism. Applied to misfiling:

- **Bad control:** relocating an entry to a random cell. This fails to preserve difficulty (the displacement is unrealistic) and so the "detection" is of distance moved, not of misfiling.
- **Better control:** relocating an entry to a semantically confusable alternative cell, preserving the displacement scale of real mistakes.

The control should kill the mechanism, not make the task impossible by fiat. Getting this backwards is itself one of the failure modes we catalog.

3 Layer A: the failure catalog

We group the twelve cases by the phase in which the error entered, and record for each the type of control that caught it and when detection occurred. Table 1 is the full catalog; the subsections give the cases in enough detail to be re-used.

Table 1: The twelve cases. **CT** is control type; **Det** is detection status.

#	Claim	Apparent positive	Control	Corrected / CT / Det
1	Geometry certifies misfiling (E14)	AUC 0.880	nearest-cell injection	0.506 mechanism null detected after
2	Antonym rule detects contradiction (E83)	recall 0.996	hand-written fixtures with no lexical antonym	0.143 adversarial fixture detected after
3	Structured rules precise on contradictions (E85)	precision 1.000	fixtures built to fire each rule wrongly	0.000 adversarial fixture detected after
4	Five congruent domains aid abstraction (E19b)	C5 − A1 = +0.367	coupled-domain arm B5c	control invalid at 4B, valid at 9B negative control retrospectively
5	Cline extraction is reliable (E74)	failure rate 0.005	inspect shell-action output	0.070 measurement sanity check detected during
6	Corpus holds 402 sessions (E73)	402 sessions	scan for byte-identical session ids	188 duplicates dropped replication check detected during
7	File revisions form a tight region (E78)	within-object distance 0.0000	check embedding covers whole file	whole-revision chunking; 0.0118 implementation invariant detected during
8	Coherence rises after perturbation (E76)	4.0 → 5.0	check which field the parser reads	parser read node count measurement sanity check detected during
9	ANCHORVOC is not circular (E17)	tests 1–2 pass	three preregistered tests, all required	test 3 fails; artifact mechanism null detected after
10	Pooled baseline scores 0.400 (E87)	0.400	rerun baseline in the same script	0.200 in-run baseline detected during
11	Static rules find no structure (E81)	ratio 0.96	decompose by cue arity	multi-word 15:1, single-word 0.92:1 measurement sanity check detected after

#	Claim	Apparent positive	Control	Corrected / CT / Det
12	The experiment is powered (E10)	pre-flight gate	power analysis	STOP returned, run proceeded power gate prevented output, not execution

3.1 Design-phase errors

Case 1 (E14). The injection distribution is the null. Covered in Section 2: 0.880 → 0.506 on a change of null alone.

Case 2 (E83). A fixture generated by inserting a WordNet antonym into a sentence produces contradictions that the antonym rule discovers by construction. On the generated set the combined antonym+negation rules reach recall 0.996 (F1 0.998). On hand-written fixtures carrying no lexical antonym, recall falls to 0.143 — one contradiction of seven. The same rule, two fixtures, two very different numbers: any figure from a same-process fixture is an upper bound. We quote recall 0.996 rather than the more flattering F1, because recall is the quantity that collapses.

Case 3 (E85). The same rule set reaching precision 1.000 on a benign holdout collapses to precision 0.000 when given fifteen consistent pairs shaped to trigger each rule. The stored artifact records five rule firings — quantifier 2, comparative 1, antonym 1, domain 1 — every one of them wrong. (The prose report of this experiment described four pairs firing; the artifact records five firings and does not record a pair count, so we cite the artifact and note the discrepancy rather than pick the tidier number.) The failure has one shape: rules match class cues without binding them to the pair's participants.

Case 4 (E19b). The control arm was invalid, and its validity depended on the model. Arm B5c reached +0.233 at 9B, where it is a valid control, but fired at 4B, 7B and 9B. Control validity is a property of the stimulus and the model together, and must be checked per model.

3.2 Implementation-phase errors

Case 5 (E74). Reading only one key of Cline's shell results yielded a failure rate of 0.005 over 8,301 actions; reading the other result keys yielded 0.070. The pipeline reported an agent that never failed because the parser never looked.

Case 6 (E73). Forty-two Claude Code sessions appeared on three machines; 18 were byte-identical copies. The corpus count was right and the session count was inflated.

Case 7 (E78). Embedding only the first 1,200 bytes of each revision produced identical vectors for twelve revisions and a within-object distance of exactly 0.0000. The "tight region" was a constant function.

Case 8 (E76). A parser selected the first number following the word "coherence" and reported a rise from 4.0 to 5.0. The number was the node count.

3.3 Interpretation-phase errors

Case 9 (E17). Three pre-registered confound tests disagreed: two passed, one failed. The first verdict reported only the first test and declared the confound resolved. Requiring all three to pass changed the conclusion. A confound check that stops at the first reassuring answer is not a check.

Case 10 (E87). A baseline of 0.400 cited from an earlier report read 0.200 when rerun in the same script. The tree had changed under the experiment.

Case 11 (E81). A static rule dictionary reported a real-to-shuffled ratio of 0.96, read as "finds no structure". Decomposed by cue arity, the two instruments inside it were opposite: multi-word cues 15:1, single-word cues 0.92:1. The aggregate hid a working instrument and a broken one.

Case 12 (E10). The pre-flight power check returned STOP. The run proceeded anyway, because the author of the check overrode it. A control its author ignores is not a control.

4 Layer B: the control taxonomy

4.1 What is a construction-matched null, and what is not

Not every safeguard is a null, and calling them all "construction-matched" empties the term. We separate them:

- **Construction-matched null** (mechanism null): preserves the construction, perturbs the claimed mechanism. Cases 1, 2, 3, 9.
- **Negative control**: an arm in which the treatment should have no effect, run to detect leakage or accidental signal. Case 4.
- **Adversarial fixture**: input deliberately constructed to trigger a rule wrongly. Cases 2, 3.
- **Implementation invariant**: an assertion about what the code must be doing, checked at runtime. Case 7.
- **Measurement sanity check**: a check that the metric reads the field it claims to read. Cases 5, 8, 11.
- **Replication check**: a redundant count or scan that a duplicate or drift would break. Case 6.
- **In-run baseline**: a baseline recomputed in the same execution rather than cited. Case 10.
- **Power gate**: a pre-flight check that the design can resolve the question. Case 12.

Only the first is a null in the strict sense. The others are safeguards, and they are not interchangeable: an implementation invariant cannot detect an invalid control, and a power gate cannot detect a parser reading the wrong field.

4.2 Detection status

Not every safeguard fires at the same point, and it matters which:

- **Prevented before execution**: Case 12, partially — the gate returned STOP and the run proceeded, so the gate prevented a verdict, not an execution. We do not count this as prevention.
- **Detected during execution**: Cases 5, 6, 7, 8, 10.
- **Detected after execution, before publication**: Cases 1, 2, 3, 9, 11.

- **Discovered retrospectively**: Case 4, where the control's validity was only established by a later model-scale comparison.

The distribution is itself a finding: five of twelve were caught only after the number existed, which is why the workflow in Section 5 makes the null a required input rather than a post-hoc review.

4.3 Claim language

We make no claim that these errors are peculiar to AI-assisted research. Circular benchmarks, measurement bugs, baseline drift, underpowered studies and selective interpretation are all long documented in human research; Section 8 cites the classics.

The defensible difference is structural, not anthropological: AI-assisted research compresses experiment design, implementation, execution and interpretation into one machine-mediated loop, so such errors can be generated and propagated rapidly and repeatedly within a single session. That is a claim about the speed and coupling of the loop, and it is the only one our evidence supports.

5 Layer C: the executable workflow

5.1 Structures

```
Claim {
  id, hypothesis, predicted_effect, metric,
  falsification_condition, control_specification,
  data_hash, code_commit, timestamp, provenance, status
}
Experiment {
  id, claim_id, code_commit, data_hash,
  treatment, controls[], preregistered_decision_rule, outputs[]
}
Verdict = Supported | Contradicted | PartiallySupported
        | Underpowered | Artifact | Superseded
RevisionEdge { previous_claim, experiment, verdict, successor_claim }
```

5.2 The invariant

No result may modify its own decision rule after observation.

This is the invariant that separates pre-registration from post-hoc rationalisation. We state its status honestly: the current system **records** violations; it does not **prevent** them. Case 12 is the proof — an agent overrode the pre-flight gate and the run proceeded. The correct description is therefore "protocol violation recorded", and an earlier draft's claim that the workflow "caught" this case overstated what happened.

This matters because a control system that is described as preventing violations will be trusted more than one described as logging them, and the difference is exactly what an agent under pressure will exploit.

5.3 Failed branch retention

The workflow's most distinctive property is that superseded claims and failed experiments stay in the record. Figure 1 shows a real lineage from the ontology track, where every step is retained and the revisions are visible.

```
claim: geometry can certify ontology membership
  |
  +-- E12  anchor audit           -> refutes the fix it was designed to prescribe
  +-- E13  anchor ceiling         -> cancels E12's own recommendation
  +-- E14  injection realism      -> 0.880 becomes 0.506; no working certifier
  +-- E15  structure survives?    -> reopens the question: CONSTRAINT 0.612
  +-- E16  real misfilings        -> E15 withdrawn; everything at chance 0.519
  +-- E17  ANCHORVOC confound     -> three tests disagree; artifact
  +-- E18  ANCHORVOC closed       -> entry-length artifact; not a certifier
  |
  v
surviving claim: geometry is unreliable for certification
  |
  +-- E21  the question reversed  -> geometry as PROPOSER: top-3 0.668
  |
  v
surviving claim: unreliable judge, useful candidate generator
```

Figure 1: The retained lineage. Every refuted intermediate survives in the repository; the surviving claim is the bottom of the chain, not a rewrite of the top.

Reading the chain downward: a claim was made, eight experiments tested it, seven failed, and each failure is preserved with its own result file. The final claim is narrower than the first and was reachable only by keeping the failures. This is the concrete form of "failures stay visible".

5.4 What the workflow caught

- **E14**: registered with a nearest-cell null; produced 0.506; marked Contradicted. Without the pre-registered null the 0.880 would have shipped.
- **E83**: registered with a fixture it did not generate; produced recall 0.143; marked Artifact.
- **E85**: registered with adversarial fixtures; produced precision 0.000; marked Contradicted.
- **E17**: registered with three tests and a rule that all must pass; test 3 failed; marked Artifact even though tests 1 and 2 passed.
- **E10**: power gate returned STOP, run proceeded; recorded as a protocol violation; marked Underpowered.

In four of these five the outcome changed from positive to negative or artifact. That is the workflow's value, and it is also its cost: it converted apparent successes into documented failures.

6 Falsifying the verifier

The apparatus this paper proposes is itself an artifact, so it gets the same treatment it prescribes for everything else. This section reports the cases where the verifier failed, because a checker with no kill tests is a comfortable way of not finding things.

6.1 The executable ledger, and attempts to violate it

The workflow's invariants are executed, not documented. The current suite is **nine invariants, zero failures** (`research/methodology_validation.json`, checked against 15 registered claims):

1. **I1 threshold frozen** — no result may change its own decision rule after observation.
2. **I2 null registered** — every claim carries a construction-matched null before it runs.
3. **I3 STOP prevents execution** — a power-gate STOP must not be followed by a run.
4. **I4 superseded addressable** — a superseded claim must still resolve to something, not vanish.
5. **I5 artifact resolves** — every claim's result file exists.
6. **I6 provenance** — every claim records a source script and a commit.
7. **I7 manuscript numbers have IDs** — no headline number lives only as hand-copied prose.
8. **I8 missing provenance fails** — validation must refuse an incomplete record.
9. **I9 artifact parameters match** — catches an artifact silently replaced by a differently-parameterised run.

I8 is the one that proves the suite can fail. A deliberately stripped record is submitted to it on every run, and it must reject it; it currently catches three violations from that probe (missing artifact file, absent `source_script`, absent `git_commit`). Without that probe, "zero failures" would be indistinguishable from a suite that never fires.

I3 is the invariant the workflow cannot satisfy. Case 12 overrode the pre-flight gate and the run proceeded; the item above records the violation rather than preventing it. The suite is honest about this: I3 reports `executed: false` against the live ledger, not an enforcement count.

6.2 Where the verifier itself was wrong

Four failures, each found by using the thing rather than reviewing it:

1. **A stale artifact overwrote a corrected one.** A fixed output path plus a long-running process is a race: the process holds the code it started with, so a late-finishing run writes the *old* verdict over the corrected artifact. This happened **four times in one session** on the Empedocles finite-size sweep, and once it printed the opposite conclusion. The fix was not a faster editor but an overwrite guard: the artifact now records `classifier_version`, and a process running an older classifier refuses to replace a newer artifact, writing a side file instead.

2. **The interpreter contradicted its own numbers.** The first version of that same classifier emitted "consistent with finite-size critical scaling" while the diagnostics it had just computed showed the susceptibility peak sitting at the edge of the parameter sweep and a transition width that *broadened* with system size. Both facts are incompatible with the verdict the classifier printed. The classification logic was rewritten (version 2, which tests the boundary condition explicitly and returns "NO TRANSITION LOCATED"). This is the cleanest available example of why a machine-generated interpretation is a measurement like any other, and it is cross-referenced from Paper 10.

3. **The manuscript checker silently blessed a wrong number.** The script that traces Paper 4's figures back to result files contained a dead index expression (`[-1][-1] if False else ...`) that made one arithmetic derivation unreachable, so a number the paper had withdrawn (`+0.576`) was still being accepted as explained. The checker reported success. It was only found by asking where a specific printed number came from. Its replacement asserts the derivation instead of tolerating it.

4. **The new checker had two bugs of its own.** The cross-rater consistency checker was wrong on first run in two independent ways: it counted distinct entry names over the wrong row subset, and it built a rectangular contingency matrix for a chance-agreement term that requires a square one (the two raters use 57 and 42 categories respectively). Both were caught by running it, not by reading it, and both are now pinned by assertions.

6.3 Provenance that is present and wrong

An invariant can pass while the record it checks is stale. The ledger snapshot used above records commit `8232042`; the repository has since moved on. Nothing in the suite fails on this, because the invariant checks that provenance *exists*, not that it is *current*. That gap is deliberate and named: a claim whose artifact predates the last relevant code change should be re-run, and detecting that requires a staleness relation the ledger does not yet carry.

6.4 The principle

A verifier needs its own kill tests, in the same sense that a classifier needs a positive control. Concretely:

- every invariant must have a case that makes it fail, exercised on every run (I8's stripped-record probe);
- every artifact path must be version-guarded against being overwritten by an older producer;
- every derived number in a manuscript must re-derive, not merely reconcile to a tolerance;
- every automated verdict must be compared against the numbers that produced it, because those are what the reader can check.

The Empedocles control battery (Paper 10) makes the same point from the other side: an instrument that cannot fire is not a falsifier. This section is the claim that the apparatus generating that control battery is subject to the same rule.

7 The empirical basis

The catalog is not assembled by inspection from memory. Each case resolves to a stored result file, and where a result file had been lost it was regenerated before being relied on. Two of the E15–E18 experiments had shipped as printing scripts with no saved output, so their numbers resolved to nothing; both were re-run from the surviving corpus and ground truth and reproduced their reported values (COSINE 0.882 under random injection and 0.519 on real misfilings; CONSTRAINT 0.612 and 0.536). The re-run is stored in `research/e15_e18_rerun.json`.

Two claims in the original reports remain unreproducible because their experiments have no saved output anywhere on disk (E5's 0.447 and E24's 0.712). They are not load-bearing here and we do not rely on them.

This is the workflow applied to the paper that describes the workflow: the count of unreproducible claims went from five to two, and the two that remain are named.

8 Related work

Exploratory versus confirmatory analysis. The requirement to separate hypothesis-generating from hypothesis-testing analysis is long-established [Tukey(1980)], and the cost of undisclosed analytic flexibility is well documented [Simmons et al.(2011)]. Our contribution is not the principle but its automation: the null is a required input to the execution rather than a review criterion applied afterwards.

Autonomous research systems. The AI Scientist [Lu et al.(2024)] implements an end-to-end loop that generates hypotheses, runs experiments and writes up results. The Robot Scientist [King et al.(2004)] automates hypothesis generation and experimentation in a laboratory, with hypotheses formalised in logic. Both describe the loop; neither describes the falsification infrastructure — how the null is specified before running, how a control's validity is itself checked, and how failed branches are retained. Our contribution is that infrastructure.

Belief revision and truth maintenance. Our claim/verdict/revision records are *AGM-inspired* and *related to* the belief-revision framework of Alchourrón and Makinson [Alchourrón & Makinson(1982)], and our justification-and-retraction structure is related in spirit to Doyle's truth maintenance system [Doyle(1979)]. We do not claim to implement either. The AGM postulates are formal properties of contraction and revision functions on belief sets; we have not shown that our revision edges satisfy them, and our objects are empirical claims carrying effect sizes and nulls, not logical propositions. The relationship is that of an operational analogue, and we state it as such.

Distribution fitting as a control. One of our cases turns on distinguishing a power law from a lognormal, for which we use standard methods [Clauset et al.(2009)]. We mention it because a heavy-tailed-looking sample is the most common route by which a null result is mistaken for a positive one, and because a fit compared against only one alternative family is not a test.

9 Position against the current verification-gap literature

A recent survey of autonomous research agents codes 26 systems across seven audit dimensions and finds that in 24 runnable systems, **83% release code while only 38% release the seeds or execution traces needed to reproduce a run, and 38% report any novelty-verification method**; among nine closed-loop autonomous systems, seven are mechanical reruns and none demonstrates an externally validated in-loop oracle [Ding et al.(2026)]. A companion position paper argues that verification, attribution and reproducibility infrastructure break down at agent

scale [Mo(2026)], and cites four observed failure modes in AI-scientist systems: benchmark cherry-picking, data leakage, metric misuse and post-hoc selection bias.

The present work is best read as a concrete instance of that gap and a partial answer to it at the scale of one programme. The cases above are exactly the failure modes that literature names — leakage (the withdrawn Empedocles cold-start curve), post-hoc selection (Case 12's override of the pre-flight gate), and metric misuse (the classifier that reported critical scaling against its own numbers) — but each is recorded with an artifact, a verdict, and a retained failed branch, which is the disclosure the survey reports as rare.

We do not claim to close the gap. Our ledger holds twelve cases from one programme and is maintained by the same agent that runs the experiments. What the survey's coding dimensions do not audit is whether a released verifier was itself falsified; Section 6 is an attempt to answer that question for this one.

10 Limitations

No control group. We cannot claim the workflow produces better research than research without it. That would require two groups using different workflows on comparable problems, which we have not run. Every statement here is about what the workflow recorded, not about comparative outcomes.

Agent compliance. The workflow catches errors only if the agent follows it, and Case 12 shows an agent can bypass it. It records the violation; it does not prevent it.

One programme, one domain. All twelve cases come from one research programme, one user, one machine fleet, over roughly three months. The frequency of these failure modes elsewhere is unknown, and the taxonomy's coverage of failure modes we did not happen to hit is unknown.

Selection. The cases are ones where a control *caught* something. We have no systematic study of cases where a control failed to catch an error a human would have caught, nor of errors that survived to publication.

Detection-status coding. The phase and timing labels in Table 1 are our own judgments, made retrospectively. They are not independently coded.

Novelty. Each individual practice here is old. Pre-registration, negative controls, power analysis, adversarial testing, in-run baselines and retaining failed results are all standard, and a reviewer is right to say so. We claim no invention of any of them. What we claim is narrower: a precise definition of the construction-matched null that distinguishes it from seven other safeguards commonly called "controls"; twelve documented cases from one programme with the correction chain intact; and an honest accounting of which safeguards detected which errors, showing that five of twelve were caught only after the number existed. If the reader's question is "is this new?", the answer is no; if it is "what does it look like when an AI-driven loop actually goes wrong, and what catches it?", we have twelve answers with stored artifacts.

11 Conclusion

Twelve documented cases from one research programme show a consistent shape: a strong positive result produced by a loop that also produced the error, and a control that killed the positive. From those cases we derived a precise definition of the construction-matched null, a taxonomy that distinguishes a null from the seven other safeguards it is commonly confused with, and an executable workflow whose records make the sequence claim → experiment → verdict → revision auditable.

The workflow's most valuable property is the least glamorous: it retains failed branches. The ontology lineage in Figure 1 reaches a defensible claim only because eight refuted intermediates were kept. If they had been deleted — or never written down — the surviving claim would have looked like a first guess rather than the end of a search.

We state plainly what this is not: not a proof that AI-assisted research is worse than human research, not an implementation of AGM belief revision, and not a prevention system. It is an instrumentation proposal with twelve worked examples, and its main result is that in four of the five cases where it was applied end-to-end the apparent positive conclusion was overturned, reclassified as an artifact, or rendered non-promotable — by a control, a pre-registered null, a provenance failure or a protocol violation, and never by a change of preference. We avoid the phrase "the answer changed sign" because the outcomes are not one kind of event: an artifact, an underpowered run, a protocol violation and a reclassification are different failures and conflating them would hide which safeguard did the work.

References

[Ding et al.(2026)] Ding, T., Nannapaneni, A., Liu, B., & Zhang, L. (2026). Autonomous research agents: a survey of AI scientists and the verification gap. arXiv:2608.05179.

[Mo(2026)] Mo, B. (2026). Position: the age of AI agents demands a new scientific paradigm to sustain trustworthy science. arXiv:2607.26064.

[Alchourrón & Makinson(1982)] Alchourrón, C. E., & Makinson, D. C. (1982). On the logic of theory change: Contraction functions and their associated revision functions. *Theoria*, 48(1), 14–37. DOI: 10.1111/j.1755-2567.1982.tb00480.x.

[Clauset et al.(2009)] Clauset, A., Shalizi, C. R., & Newman, M. E. J. (2009). Power-law distributions in empirical data. *SIAM Review*, 51(4), 661–703. DOI: 10.1137/070710111.

[Doyle(1979)] Doyle, J. (1979). A truth maintenance system. *Artificial Intelligence*, 12(3), 231–272. DOI: 10.1016/0004-3702(79)90008-0.

[King et al.(2004)] King, R. D., Whelan, K. E., Jones, F. M., Reiser, P. G. K., Bryant, C. H., Muggleton, S. H., Kell, D. B., & Oliver, S. G. (2004). Functional genomic hypothesis generation and experimentation by a robot scientist. *Nature*, 427(6971), 247–252. DOI: 10.1038/nature02236.

[Lu et al.(2024)] Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., & Ha, D. (2024). The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint* arXiv:2408.06292.

[Simmons et al.(2011)] Simmons, J. P., Nelson, L. D., & Simonsohn, U. (2011). False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11), 1359–1366. DOI: 10.1177/0956797611417632.

[Tukey(1980)] Tukey, J. W. (1980). We need both exploratory and confirmatory. *The American Statistician*, 34(1), 23–25. DOI: 10.1080/00031305.1980.10482706.